Municipal air-quality readings stored in retryable batches

Purpose and data

The system feeds a city dashboard: planners compare hourly readings by station and date, analysts review trends and gaps, and developers query one stable data source. It issues no health alerts. As sample data I use the UCI dataset Beijing Multi-Site Air Quality, which holds 420,768 hourly records from 12 stations with pollutant and weather fields (UCI Machine Learning Repository, n.d.). This is close to the half-million-reading scenario.

Database choice

MongoDB stores each station-hour as one document with fixed shared fields and a flexible measurements object. Future sensors can add metrics without a schema migration, while validation protects station, timestamp and source (MongoDB, n.d.-a). PostgreSQL suits fixed schemas and join-heavy work; plain CSV files offer no indexed concurrent queries; an analytical platform would add setup effort too early.

Reliability, scalability, maintainability

Deterministic IDs built from station and time, insert-only upserts and bounded retries make a replayed batch harmless. Missing values stay absent instead of becoming zero. Docker Compose keeps the prototype portable. Replica sets and sharding are later options, and sharding adds operational complexity (MongoDB, n.d.-b).

Plan and success criteria

Define the document shape, initialise MongoDB with a station/time index, load batches of 1,000, test reruns, then verify the full containerised run. Success means complete, duplicate-free station/time queries; misread timestamps or dropped fields count as failure.

References

UCI Machine Learning Repository. (n.d.). *Beijing Multi-Site Air Quality*. https://archive.ics.uci.edu/dataset/501/beijing+multi+site+air+quality+data

MongoDB. (n.d.-a). *Data modeling*. https://www.mongodb.com/docs/manual/data-modeling/

MongoDB. (n.d.-b). *Sharding*. https://www.mongodb.com/docs/manual/sharding/